Question 1 — What a proper architecture gives you

1. Simplicity (KISS)
The `MainActivity.kt` class has too many responsibilities, for example, if I want to find a UI bug I have to look through 600 lines of code including server requests and JSON parsing. This makes the code difficult to understand and change.

2. Testability
It is difficult to test one part separately because everything is connected in one class, for example, if I want to test form validation I may also have to deal with server requests and other code. If validation is separate, testing becomes much easier.

3. Low-cost maintenance
If I change one part of the code it may affect other parts for example, if the server changes its JSON format, I may need to modify `MainActivity.kt`, this can cause new bugs and take more time.

Which piece would I move first, and why?

First I would move server communication and JSON parsing into a separate class. I think this is a good starting point because `MainActivity` should mainly handle the UI. It would make the code cleaner and easier to test and maintain.

Question 2 — Starting from Nothing

First of all, I would study the existing app and understand how its different parts work together.

Then, I would learn the main architectural patterns, such as MVC, MVP, MVVM, and MVI.

After that, I would choose a pattern that fits the project's needs.

I would apply the pattern to one feature first instead of changing the whole application.

Finally, I would check whether the code has become easier to understand, test, and maintain.

Why should understanding the app come first?

Before choosing a pattern, I need to understand the project's structure and requirements. Otherwise, I might choose a pattern that makes the code more complicated instead of solving its problems.

Why is “use MVVM everywhere” not good advice?

I think, there is no single pattern that works best for every project. MVVM can be a good choice for Android development but other patterns may work better in some situations. Also different modules in the same project can use different structures. The important thing is to understand the project and choose the pattern that fits its needs.

Question 5 — MVC Roles and the Missing Controller

1. Assign each piece to Model, View, or Controller.

1. `Weather` data class — Model
2. `WeatherRepository` — Model
3. `WeatherScreen` composable — View
4. `onClick` lambda in `MainActivity` — Controller and View
5. `FakeWeatherApi` — Model
6. `WeatherController` — Controller

2. Why is piece 4 a problem?

Piece 4 plays two roles at the same time: Controller and View. It handles the user’s click and calls the repository, which is Controller work. It also updates the `TextView`, which is View work.

As a result, `MainActivity` becomes larger and more complicated. Testing is also harder because the UI logic and application logic are mixed together.

3. Complete the sentence.

The Model has no knowledge of the View.

Question 7 — MVP Rules: Find the Violations

1. Which rules do lines (A) and (B) break?

> Line (A): It breaks Rule 1 because the View accesses the Model directly by calling `repository.getWeather("Tashkent")`. The Presenter should handle this work.
>
> Line (B): It breaks Rule 3 because the Activity updates the UI itself instead of letting the Presenter control the process.

2. What is wrong with lines (C) and (D)?

Lines (C) and (D) do not directly break the three rules, but they make testing difficult. Line (C) passes the real `WeatherActivity` to the Presenter, while line (D) makes the Presenter depend directly on that Activity. Therefore, testing the Presenter separately requires an Activity, making unit testing harder.

The code respects Rule 2 because one Presenter is connected to one View, `WeatherActivity`.

`WeatherPresenter`**3.** Rewrite the constructor parameter of **.**

Instead of passing the Activity directly, I would use a View interface:

```
interface WeatherView {

    fun showLoading()

    fun showWeather(weather: String)

    fun showError()

}


class WeatherPresenter(

    private val view: WeatherView

)
```

This way, the Presenter depends on an interface rather than a specific Activity. It becomes easier to test using a fake implementation of `WeatherView`.

Question 10 — MVVM: Where Does Each Class Live?

1. Put each class into the correct package

Class		Package
`WeatherApi`		`data/api`
`FakeWeatherApi`		`data/api`
`Weather`		`data/model`
`WeatherRepository`		`data/repository`
`WeatherModule`		`di`
`WeatherViewModel`		`presentation/viewmodel`
`WeatherUiState`		`presentation/viewmodel`
`WeatherScreen`		`presentation/view`
`MvvmActivity`		`presentation/view`

2. (a) Why must the ViewModel not hold a reference to the Activity or a composable?

The ViewModel should not depend directly on the Activity or composable because it makes the code harder to test and maintain. Also, the Activity can be recreated during configuration changes, which may cause memory leaks if the ViewModel keeps a reference to it.

Instead, the ViewModel exposes UI state and functions that the View can observe and use. For example, it can expose `WeatherUiState` through a `StateFlow`, and the composable can observe it and update the UI.

3. (b) Give one concrete test for the ViewModel.

I can write a unit test to check whether `WeatherViewModel` correctly displays weather data when the repository returns a successful result.

For example, I can use a fake repository that returns sunny weather and check whether the ViewModel updates its UI state with the correct weather information.

This test does not require an Activity or a composable because the ViewModel is independent of the UI.

Question 12 — MVI Principles: Spot the Violations

1. Match each spot to the principle it breaks

(1) Separation of concerns: The ViewModel stores several independent mutable variables instead of managing the weather information as one state object.

(2) Unidirectional data flow: The View calls separate functions directly instead of sending user actions as Intents through one public function.

(3) Immutability: `WeatherState` uses `var`, so its properties can be changed after the state is created; it should use `val`.

(4) Separation of concerns: The Activity changes the ViewModel's state directly instead of sending a user action to the ViewModel to process.

Note: Spot (4) also breaks unidirectional data flow, but separation of concerns is the better choice because the Activity is directly modifying state that should be managed by the ViewModel.

2. Write the three declarations required for MVI

```
data class WeatherState(
    val city: String = "Tashkent",
    val isLoading: Boolean = false,
    val temperatureC: Int = 0,
    val error: String? = null
)


sealed interface WeatherIntent {
    data class CityChanged(val name: String) : WeatherIntent
    data object Refresh : WeatherIntent
}


class WeatherViewModel : ViewModel() {
    fun onIntent(intent: WeatherIntent)
}
```

Explanation:

`WeatherState` holds the complete UI state using immutable properties (`val`).

`WeatherIntent` represents actions requested by the user.

`onIntent()` is the single public function through which the View sends user actions to the ViewModel.